	School of Engineering & Technology	
	Department: SOET	Session: 2026-2027
	Program: B.Tech/ BCA/ BSc	Semester:3rd
	Course Code: ETCCDA302/ ETCCDA301/ ETCCAD301	Number of students:
	Course Name: Design and Analysis of Algorithms	Faculty: Ms. Ritu Devi

Question Bank: Unit-2

Short Answer Type Questions

1. Explain the three steps of the divide-and-conquer strategy (divide, conquer and combine) with reference to Merge Sort.
2. Why is the divide-and-conquer paradigm considered more efficient than brute force for large problem sizes?
3. How does the choice of pivot affect the performance of Quick Sort?
4. Why does Merge Sort guarantee the same time complexity in the best, average and worst cases?
5. Why is Merge Sort preferred over Quick Sort for external sorting?
6. What property of Quick Sort makes it an in-place sorting algorithm? Is Quick Sort stable?
7. Why is Binary Search unsuitable for unsorted data, and why does it require random access to elements?
8. Write the recurrence relation for Binary Search and solve it to obtain its time complexity.
9. Define the greedy choice property and optimal substructure. How do they differ?
10. Why do greedy algorithms not always guarantee an optimal solution? Support your answer with an example.
11. Why does the fractional knapsack problem satisfy the greedy choice property while the 0/1 knapsack problem does not?
12. How does selecting activities in order of earliest finishing time lead to the maximum number of compatible activities?
13. In job sequencing with deadlines, why is each selected job placed in the latest free time slot on or before its deadline?
14. What are overlapping subproblems? Illustrate with the computation of Fibonacci numbers.
15. Distinguish between memoization and tabulation in dynamic programming.
16. Why is tabulation sometimes preferred over memoization in dynamic programming?
17. Why is the dynamic programming approach effective for solving the Longest Common Subsequence problem?
18. What is the significance of optimal substructure in Matrix Chain Multiplication?
19. Why does the order of matrix multiplication affect the computational cost but not the final result?
20. Give three differences between the divide-and-conquer and dynamic programming

approaches.

21. Consider the following algorithm. How many calls are made to compute Fib(5)? What is its time complexity?

```
Algorithm Fib(n)
    if n ≤ 1 then
        return n
    end if
    return Fib(n − 1) + Fib(n − 2)
end Fib
```

Long Answer Type Questions

1. The following divide-and-conquer algorithm finds the largest element of an array A[low..high].

```
Algorithm DCMax(A, low, high)
    if low = high then
        return A[low]
    end if
    mid ← ⌊(low + high) / 2⌋
    leftMax ← DCMax(A, low, mid)
    rightMax ← DCMax(A, mid + 1, high)
    if leftMax > rightMax then
        return leftMax
    else
        return rightMax
    end if
end DCMax
```

   (a) Identify the divide, conquer and combine steps of the algorithm.
   (b) Write the recurrence relation for the number of element comparisons and solve it.
   (c) Compare the result with the straightforward iterative method of finding the maximum.

2. A telecom company stores call records of millions of customers by call duration. These records must be sorted in ascending order to analyse network usage patterns.
   (a) Explain why **Merge Sort** is suitable for this application.
   (b) Write the Merge Sort algorithm in pseudocode and apply it to the following sample call durations: [320, 45, 600, 120, 90, 30]
   (c) Derive the time complexity of Merge Sort.

3. A gaming company needs to rank players based on their scores in real time. The scores are frequently updated. Quick Sort with the following partition procedure is used.

```
Algorithm Partition(A, p, r)
   x ← A[r]       // pivot is the last element
   i ← p − 1
   for j ← p to r − 1 do
      if A[j] ≤ x then
         i ← i + 1
         swap(A[i], A[j])
      end if
   end for
   swap(A[i + 1], A[r])
   return i + 1
end Partition

Algorithm QuickSort(A, p, r)
   if p < r then
      q ← Partition(A, p, r)
      QuickSort(A, p, q − 1)
      QuickSort(A, q + 1, r)
   end if
end QuickSort
```

   (a) Justify the use of **Quick Sort** for this scenario.
   (b) Sort the scores [450, 1200, 300, 980, 700, 1500] using the above algorithm, showing the array after every call to Partition.
   (c) State the best-case and worst-case time complexities of Quick Sort, and comment on the first partition obtained in part (b).

4. A library management system stores book IDs in sorted order. When a student searches for a book, the system must respond quickly.
   (a) Explain how **Binary Search** improves search efficiency and write the iterative Binary Search algorithm in pseudocode.
   (b) Find book ID 2456 in the list [1020, 1345, 1789, 2001, 2456, 2890, 3100], showing the values of low, high and mid at each step.
   (c) An online exam portal searches for a student's registration number in a sorted database of 1 million records. Calculate the maximum number of comparisons required using Binary Search.

5. Write the greedy algorithm for the **Fractional Knapsack** problem and find the optimal solution for the following instance with knapsack capacity M = 20:

Item	I1	I2	I3	I4	I5
Profit (P)	15	5	20	8	7
Weight (W)	3	4	6	8	2

6. Given n activities with their start and finish times, select the maximum number of activities that can be performed by a single person, assuming that a person can work on only one activity at a time:

Activity	A3	A2	A1	A5	A6	A4
Start	1	3	0	5	8	5
Finish	2	4	6	7	9	9

7. An array of jobs is given, where every job has a deadline and an associated profit if the job is

finished before the deadline. Every job takes a single unit of time, so the minimum possible deadline for any job is 1. Find the sequence of jobs that maximises the total profit if only one job can be scheduled at a time.

Jobs	J1	J2	J3	J4	J5	J6	J7
Profits	35	30	25	20	15	12	5
Deadlines	3	4	4	2	3	1	2

8. A message of 100 characters contains only the characters a–f with the following frequencies:

Character	a	b	c	d	e	f
Frequency	5	9	12	13	16	45

   (a) Construct the Huffman tree.
   (b) Write the Huffman code for each character.
   (c) Find the total number of bits needed to encode the message and compare it with a fixed-length code.

9. You are given a knapsack with a capacity of W = 8 kg and 4 items, each with a specific weight and profit as shown below:

Item	A	B	C	D
Profit	2	4	7	10
Weight	1	3	5	7

   Select a subset of these items such that the total weight does not exceed 8 kg and the total profit is maximised. Use the dynamic programming approach to:
   (a) Formulate the recurrence relation.
   (b) Construct the DP table.
   (c) Determine the maximum profit.
   (d) List the items included in the optimal subset.

10. For the given set of items and a knapsack capacity of 10 kg, find the subset of the items to be added to the knapsack such that the profit is maximum (0/1 knapsack, dynamic programming).

Items	1	2	3	4	5
Weights (in kg)	3	3	2	5	1
Profits	10	15	10	12	8

11. Consider a knapsack of capacity W = 7 and the following 5 items:

Items	I1	I2	I3	I4	I5
Profits	8	12	18	22	20
Weights	1	4	2	3	5

   (a) Find the maximum profit if items can be broken (fractional knapsack, greedy method).
   (b) Find the maximum profit if items cannot be broken (0/1 knapsack, dynamic programming).
   (c) Explain why the greedy method cannot be relied upon for the 0/1 knapsack problem, with a suitable example.

12. Use dynamic programming to find the **Longest Common Subsequence** of the following two sequences:

    S1 = {B, C, D, A, A, C, D} S2 = {A, C, D, B, A, C}

13. Find the **Longest Common Subsequence** of the strings X = BACDB and Y = BDCB using dynamic programming. Show the table with direction arrows and trace back the LCS.

14. A sequence of matrices A, B, C, D with dimensions 5×10, 10×15, 15×20 and 20×25 is to be multiplied. Find the lowest cost of parenthesization using matrix chain multiplication.

15. Given a sequence of matrices A1, A2, A3, A4 with dimensions A1 = 10×30, A2 = 30×5, A3 = 5×60 and A4 = 60×10:
    (a) Formulate the dynamic programming recurrence relation for the Matrix Chain Multiplication problem.
    (b) Construct the DP table to compute the minimum number of scalar multiplications required.
    (c) Determine the optimal parenthesization of the matrix chain.
    (d) State the minimum number of scalar multiplications needed.

16. Compare the divide-and-conquer, greedy and dynamic programming techniques with respect to their approach, nature of subproblems, optimality, efficiency and typical applications. For each of the following problems, state the most suitable technique and justify your choice: sorting n numbers, fractional knapsack, 0/1 knapsack, activity selection, longest common subsequence and searching in a sorted array.